

Otimização

6.4 Método do ε -constrangimento

José Firmino Aguilar Madeira

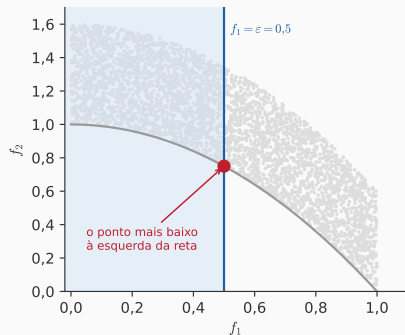
Outra ideia: otimizar um, limitar o outro

No 6.3, com $f_1 = x$, $f_2 = 1 - x^2$, a soma ponderada só encontrava os dois extremos. E se, em vez de combinar os objetivos, limitarmos um deles?

$$\min_{x \in X} f_2(x) \quad \text{s.a.} \quad f_1(x) \leq \varepsilon$$

Entre as soluções com f_1 aceitável, a de menor f_2 .

No espaço dos objetivos: a reta vertical $f_1 = \varepsilon$ e o ponto mais baixo à sua esquerda.



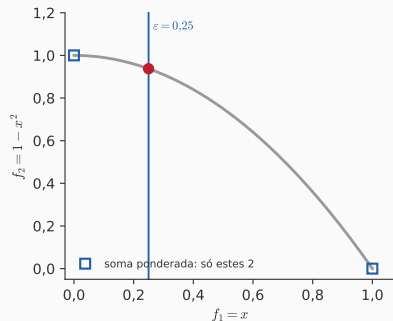
O exemplo não convexo do 6.2 (Y a cinzento).

O mesmo exemplo do 6.3, à mão

$\min f_2 = 1 - x^2$ s.a. $f_1 = x \leq \varepsilon$, $0 \leq x \leq 1$.

Como $1 - x^2$ diminui quando x aumenta: $x^* = \varepsilon$.

$$\varepsilon = 0,25 \Rightarrow (f_1, f_2) = (0,25; 0,9375)$$



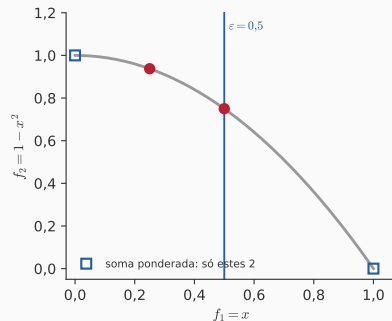
O mesmo exemplo do 6.3, à mão

$\min f_2 = 1 - x^2$ s.a. $f_1 = x \leq \varepsilon$, $0 \leq x \leq 1$.

Como $1 - x^2$ diminui quando x aumenta: $x^* = \varepsilon$.

$\varepsilon = 0,25 \Rightarrow (f_1, f_2) = (0,25; 0,9375)$

$\varepsilon = 0,50 \Rightarrow (f_1, f_2) = (0,50; 0,75)$



O mesmo exemplo do 6.3, à mão

$\min f_2 = 1 - x^2$ s.a. $f_1 = x \leq \varepsilon$, $0 \leq x \leq 1$.

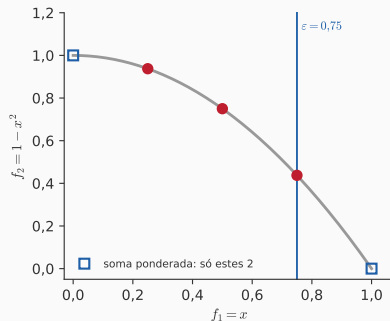
Como $1 - x^2$ diminui quando x aumenta: $x^* = \varepsilon$.

$\varepsilon = 0,25 \Rightarrow (f_1, f_2) = (0,25; 0,9375)$

$\varepsilon = 0,50 \Rightarrow (f_1, f_2) = (0,50; 0,75)$

$\varepsilon = 0,75 \Rightarrow (f_1, f_2) = (0,75; 0,4375)$

Estes três pontos eram impossíveis de obter pela soma ponderada.



O mesmo exemplo do 6.3, à mão

$\min f_2 = 1 - x^2$ s.a. $f_1 = x \leq \varepsilon$, $0 \leq x \leq 1$.

Como $1 - x^2$ diminui quando x aumenta: $x^* = \varepsilon$.

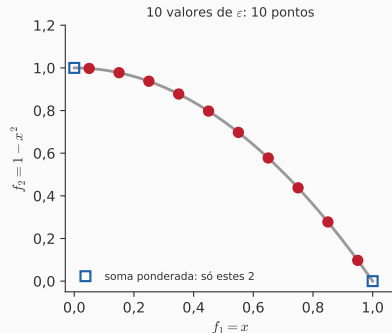
$$\varepsilon = 0,25 \Rightarrow (f_1, f_2) = (0,25; 0,9375)$$

$$\varepsilon = 0,50 \Rightarrow (f_1, f_2) = (0,50; 0,75)$$

$$\varepsilon = 0,75 \Rightarrow (f_1, f_2) = (0,75; 0,4375)$$

E se continuarmos a deslocar a reta?

$\varepsilon = 0,05; 0,15; \dots; 0,95$: a frente fica praticamente percorrida.



O mesmo exemplo do 6.3, à mão

$\min f_2 = 1 - x^2$ s.a. $f_1 = x \leq \varepsilon$, $0 \leq x \leq 1$.

Como $1 - x^2$ diminui quando x aumenta: $x^* = \varepsilon$.

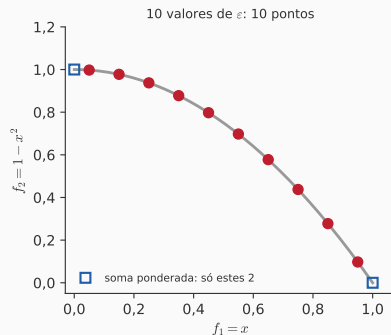
$\varepsilon = 0,25 \Rightarrow (f_1, f_2) = (0,25; 0,9375)$

$\varepsilon = 0,50 \Rightarrow (f_1, f_2) = (0,50; 0,75)$

$\varepsilon = 0,75 \Rightarrow (f_1, f_2) = (0,75; 0,4375)$

E se continuarmos a deslocar a reta?

$\varepsilon = 0,05; 0,15; \dots; 0,95$: a frente fica praticamente percorrida.



soma ponderada: 10 000 pesos
→ 2 pontos

vs.

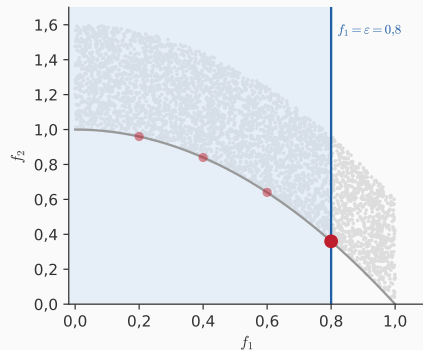
ε -constrangimento: 10 valores
de ε → 10 pontos

Porque consegue?

A soma ponderada usava uma **régua inclinada**, que não consegue tocar primeiro no interior côncavo.

Aqui a reta $f_1 = \varepsilon$ é **vertical**: pode cortar a região em qualquer posição, e escolhemos o ponto mais baixo à sua esquerda.

ε **controla diretamente quanto aceitamos em f_1** : se a restrição fica ativa, $f_1(x^*) = \varepsilon$. No 6.3, $w \rightarrow$ posição difícil de prever.



$\varepsilon = 0,2; 0,4; 0,6; 0,8$ no exemplo não convexo do 6.2.

A formulação geral

Haimes, Lasdon e Wismer (1971). Com m objetivos, escolhe-se um, que numeramos f_1 (no exemplo, era o f_2):

$$\min_{x \in X} f_1(x) \quad \text{s.a.} \quad f_i(x) \leq \varepsilon_i, \quad i \neq 1$$

- Um objetivo continua objetivo; os outros $m - 1$ passam a **restrições**: é um problema do cap. 4.
- Os ε_i têm **as unidades de f_i** : são limites nos objetivos, não tolerâncias numéricas.
- Varrimento: $\varepsilon_i \in [z_i^{id}, \hat{z}_i^{nad}]$ (6.2). Abaixo de z_i^{id} : inadmissível.

**Nantes–Lille: o mais barato,
com duração $\leq \varepsilon$**

$\varepsilon = 4$ h avião (134 €)

$\varepsilon = 5$ h TGV low-cost (40 €)

$\varepsilon = 10$ h autocarro 2 (28 €)

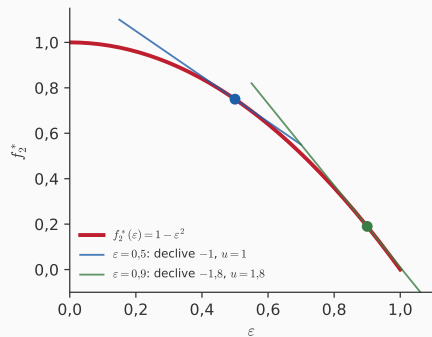
ε tem **significado físico**: muitas vezes é mais fácil para o decisor do que um peso.

O multiplicador: a taxa de troca

No exemplo, $f_2^*(\varepsilon) = 1 - \varepsilon^2$. Na forma da UC (4.2), $g = \varepsilon - f_1 \geq 0$, $\mathcal{L} = f_2 - u g$, $u \geq 0$, e o multiplicador de $f_1 \leq \varepsilon$ é

$$u = 2\varepsilon = -\frac{df_2^*}{d\varepsilon}$$

Em $\varepsilon = 0,5$: $x^* = 0,5$, $f_2^* = 0,75$, $u = 1$.



O multiplicador: a taxa de troca

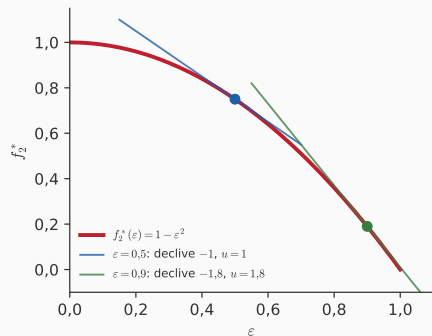
No exemplo, $f_2^*(\varepsilon) = 1 - \varepsilon^2$. Na forma da UC (4.2), $g = \varepsilon - f_1 \geq 0$, $\mathcal{L} = f_2 - u g$, $u \geq 0$, e o multiplicador de $f_1 \leq \varepsilon$ é

$$u = 2\varepsilon = -\frac{df_2^*}{d\varepsilon}$$

Em $\varepsilon = 0,5$: $x^* = 0,5$, $f_2^* = 0,75$, $u = 1$.

Se aceitarmos aumentar f_1 de 0,50 para 0,51, quanto ganhamos em f_2 ?

Com $u = 1$: $\Delta f_2^* \approx -0,01$.



O multiplicador: a taxa de troca

No exemplo, $f_2^*(\varepsilon) = 1 - \varepsilon^2$. Na forma da UC (4.2), $g = \varepsilon - f_1 \geq 0$, $\mathcal{L} = f_2 - u g$, $u \geq 0$, e o multiplicador de $f_1 \leq \varepsilon$ é

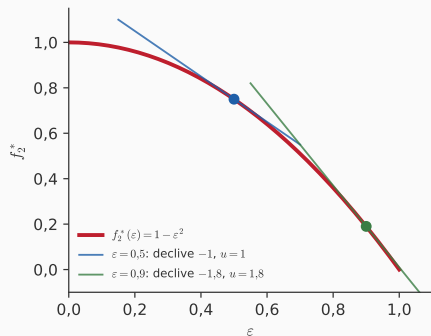
$$u = 2\varepsilon = -\frac{df_2^*}{d\varepsilon}$$

Em $\varepsilon = 0,5$: $x^* = 0,5$, $f_2^* = 0,75$, $u = 1$.

Se aceitarmos aumentar f_1 de 0,50 para 0,51, quanto ganhamos em f_2 ?

Com $u = 1$: $\Delta f_2^* \approx -0,01$.

Confirmação exata: $f_2(0,51) = 1 - 0,51^2 = 0,7399$; diferença $-0,0101$. Em $\varepsilon = 0,9$, $u = 1,8$: a mesma folga compra $\approx 0,018$.



O multiplicador: a taxa de troca

No exemplo, $f_2^*(\varepsilon) = 1 - \varepsilon^2$. Na forma da UC (4.2), $g = \varepsilon - f_1 \geq 0$, $\mathcal{L} = f_2 - u g$, $u \geq 0$, e o multiplicador de $f_1 \leq \varepsilon$ é

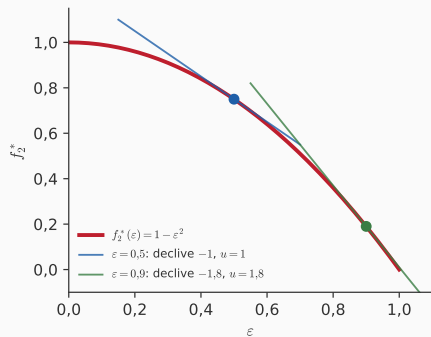
$$u = 2\varepsilon = -\frac{df_2^*}{d\varepsilon}$$

Em $\varepsilon = 0,5$: $x^* = 0,5$, $f_2^* = 0,75$, $u = 1$.

Se aceitarmos aumentar f_1 de 0,50 para 0,51, quanto ganhamos em f_2 ?

Com $u = 1$: $\Delta f_2^* \approx -0,01$.

Confirmação exata: $f_2(0,51) = 1 - 0,51^2 = 0,7399$; diferença $-0,0101$. Em $\varepsilon = 0,9$, $u = 1,8$: a mesma folga compra $\approx 0,018$.



u é a **taxa local de troca** entre objetivos — informação preciosa para o decisor.

O que podemos garantir?

Com o subproblema resolvido **globalmente**:

- se x^* é a **única** solução do subproblema, é **eficiente**;
- qualquer x^* eficiente é recuperado escolhendo $\varepsilon_i = f_i(x^*)$ ($i \neq 1$) (Chankong e Haimes, 1983) — convexo ou não.

Em princípio recupera **toda a frente**, incluindo os *non-supported* — **desde que os subproblemas sejam resolvidos globalmente**.

Sem unicidade, pode haver empates: outra solução ótima do subproblema com o mesmo f_1 e melhor num objetivo limitado. Garante-se só eficiência *fraca* (se x fosse estritamente melhor em todos, seria admissível e com menor f_1). Para eficiência: ε -constrangimento aumentado ou desempate.

O que podemos garantir?

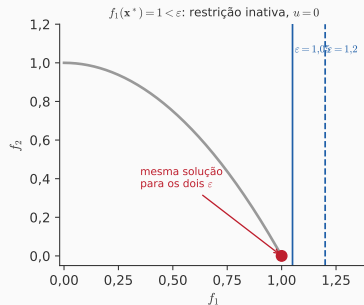
Com o subproblema resolvido **globalmente**:

- se x^* é a **única** solução do subproblema, é **eficiente**;
- qualquer x^* eficiente é recuperado escolhendo $\varepsilon_i = f_i(x^*)$ ($i \neq 1$) (Chankong e Haimes, 1983) — convexo ou não.

Em princípio recupera **toda a frente**, incluindo os *non-supported* — desde que os subproblemas sejam resolvidos globalmente.

Sem unicidade, pode haver empates: outra solução ótima do subproblema com o mesmo f_1 e melhor num objetivo limitado. Garante-se só eficiência *fraca* (se x fosse estritamente melhor em todos, seria admissível e com menor f_1). Para eficiência: ε -constrangimento aumentado ou desempate.

Restrição inativa. Se $f_i(x^*) < \varepsilon_i$, então $u_i = 0$: o limite não está a determinar a solução — mover a reta não a muda.



O preço do ε -constrangimento

Vantagens

- encontra *supported* e *non-supported* (com subproblemas resolvidos globalmente);
- ε_i com interpretação física direta;
- melhor controlo de onde procurar;
- multiplicadores = taxas de troca;
- reutiliza os métodos com restrições do cap. 4.

Limitações

- uma otimização **por ponto**;
- cada subproblema ganha $m - 1$ restrições;
- ε demasiado exigente $\rightarrow$ inadmissível;
- restrições podem ficar inativas;
- em problemas não convexos, resolver localmente os subproblemas não preserva as garantias globais;
- custo cresce depressa com m .

k valores para cada um dos $m - 1$ limites: k^{m-1} problemas $k = 10: \quad m = 2 \rightarrow 10; \quad m = 3 \rightarrow 100;$
 $m = 5 \rightarrow 10\,000.$

Em MATLAB: varrer ε com fmincon

O núcleo:

```
f1 = @(x) x(1); f2 = @(x) 1 - x(1)^2 + x(2);  
lb = [0 0]; ub = [1 0.6]; x0 = [0.5 0.3];  
info.nfev = 0;  
for e = 0.05:0.1:0.95  
    nonl = @(x) deal(f1(x) - e, []); % f1 <= e  
    [x, fval, ~, out, lambda] = fmincon(f2, x0, ...  
                                       [], [], [], [], lb, ub, nonl);  
    u = lambda.ineqnonlin; % taxa de troca  
    info.nfev = info.nfev + out.funcCount;  
    x0 = x; % a quente  
end
```

A função completa (EpsilonConstraint, com as opções, a lista dos pontos e o gráfico da frente) está no código complementar da UC.

Soma ponderada: `fmincon(@(x) w1*f1(x)+w2*f2(x), ...)`
sem `nonl`.

- `lambda.ineqnonlin`: multiplicador de $f_1 \leq \varepsilon$, aqui 2ε .
- Arranque a quente: a solução para ε é bom ponto inicial para $\varepsilon + \Delta\varepsilon$.
- `out.funcCount` conta os pontos onde se avaliam f_2 e f_1 (incluindo os dos gradientes progressivos do solver): é o n_f do varrimento. Em Python (SLSQP): 10 pontos, 60 avaliações de f_2 , 90 de f_1 (150 chamadas).

O MESMO EM PYTHON

```
minimize(f2, x0, bounds=b, constraints=  
{ 'type': 'ineq', 'fun': lambda x: e - f1(x) },  
method='SLSQP'); r.nfev conta as avaliações de f2;  
multiplicadores em r.v com trust-constr (4.2).
```

Soma ponderada e ε -constrangimento

	soma ponderada (6.3)	ε -constrangimento (6.4)
parâmetro	pesos w	limites ε
subproblema	objetivo agregado	um objetivo + $m - 1$ restrições
pontos <i>non-supported</i>	não	sim*
posição na frente	indireta	direta, se a restrição está ativa
informação extra	—	multiplicadores (taxas de troca)
custo	1 otimização por ponto	1 otimização por ponto
muitos objetivos	pesos difíceis de escolher	k^{m-1} cresce depressa

* com subproblemas resolvidos globalmente (página «O que podemos garantir?»).

O QUE DEVE FICAR — 6.4

1. $\min f_1$ s.a. $f_i \leq \varepsilon_i$ ($i \neq 1$): um objetivo é otimizado; os restantes tornam-se **limites**.
2. ε tem interpretação direta; com a restrição ativa, $f_i(x^*) = \varepsilon_i$.
3. Ao contrário da soma ponderada, encontra também pontos *non-supported* (com subproblemas resolvidos globalmente).
4. Os multiplicadores, $u_i = -\partial f_1^* / \partial \varepsilon_i$, são **taxas locais de troca**.
5. Continua a custar **uma otimização por ponto**; com m objetivos, até k^{m-1} subproblemas.

Até agora obtivemos a frente repetindo problemas escalares: um ponto de cada vez.

Podemos construir diretamente uma aproximação da frente numa única execução?

6.5 DMS: uma lista de pontos

6.6 NSGA-II: uma população

Referências Lista completa: *Bibliografia da UC*, no fim do PDF da UC.

- Haimes, Y. Y., Lasdon, L. S., Wismer, D. A., *On a bicriterion formulation of the problems of integrated system identification and system optimization*, IEEE Transactions on Systems, Man, and Cybernetics SMC-1(3):296–297, 1971.
- Chankong, V., Haimes, Y. Y., *Multiobjective Decision Making: Theory and Methodology*, North-Holland, 1983.
- Miettinen, K., *Nonlinear Multiobjective Optimization*, Kluwer Academic Publishers, 1999.

Recursos

- Código complementar: EpsilonConstraint. MATLAB: `fmincon (lambda.ineqnonlin, out.funcCount)`.
- Python: `minimize(f2, x0, constraints={'type':'ineq', 'fun':lambda x: e - f1(x)}, method='SLSQP')`; r.v com `trust-constr`.